

Week 10 Machine Learning for Regression

Discontinuity Design

Regression Discontinuity

We analyze the effect of the antipoverty program *Progresas/Oportunidades* on the consumption behavior of families in Mexico in the early 2000s.

The program was intended for families in extreme poverty and included financial incentives for participation in measures that improved the family's health, nutrition and children's education. The effect of this program is a widely studied problem in social and economic sciences and, according to the WHO, was a very successful measure in terms of reducing extreme poverty in Mexico.

Eligibility for the program was determined based on a pre-intervention household poverty-index. Individuals above a certain threshold received the treatment (participation in the program) while individuals below the threshold were excluded and recorded as a control group. All observations above the threshold participated in the program, which makes the analysis fall into the standard (sharp) regression discontinuity design.

```
## load packages
library(rdrobust)
library(rddensity)
library(fastDummies)
library(randomForest)
library(hdm)
library(gbm)
## install.packages("https://cran.r-project.org/src/contrib/Archive/rdd/rdd_0.57.tar.gz")
library(rdd)
```

Read Data

```
## load data
df <- read.csv("https://raw.githubusercontent.com/CausalAIBook/MetricsMLNotebooks/main/
               row.names = 1)
comp <- complete.cases(df)
df <- df[comp, ]
```

The data set contains 1,944 observations for which full covariate information of 27 variables is available. We want to measure the local average treatment effect of program participation on four outcome variables. The outcome variables are food and non-food consumption of the recorded families at two points in time, one year and two years after the implementation of the program.

The baseline covariates, recorded prior to program implementation, include the household's size; household head's age, sex, years of education and employment status; spouse's age and years of education; number of children not older than five years and their sex, and physical characteristics of the house: whether the house has cement floors, water connection, water connection inside the house, a bathroom, electricity, number of rooms, pre-intervention consumption, and an identifier of the urban locality in which the house is located.

Estimation Without Covariates

First, we will perform a very simple RD estimation with a weighted linear regression. We use a triangular kernel, which assigns weights to observations based on their distance from the cutoff point. The weights decrease linearly as the distance from the cutoff point increases.

```
## triangular kernel
triangular_kernel <- function(index, h) {
  weights <- 1 - abs(index) / h
  weights[weights < 0] <- 0
  return(weights)
}
```

The parameter h is the bandwidth that controls the range of observations that receive non-zero weights. We use the `IKbandwidth` function from the `rdd` package that implements the *Imbens-Kalyanaraman* method.

$$h_{IK} = C_K \left[\frac{\sigma_+^2 + \sigma_-^2}{n f(c) (m_+''(c) - m_-''(c))^2} \right]^{1/5}$$

where C_K is a constant depending on the kernel function. For the triangular kernel, $C_K = 3.4375$.

```
## Imbens-Kalyanaraman bandwidth
h <- IKbandwidth(X = df$index, Y = df$conspcfood_t1, cutpoint = 0)
```

We use the triangular kernel function to calculate weights for each observation. After that, we can fit two separate linear regressions for both treatment and control groups.

```
## manually calculate \tau at cutoff
weights <- triangular_kernel(df$index, h)
model_treated <- lm(conspcfood_t1 ~ index, data = df[df$index > 0, ], weights = weights)
model_control <- lm(conspcfood_t1 ~ index, data = df[df$index < 0, ], weights = weights)

## predict counterfactual at cutoff
cutoff <- 0
treatment_effect <- predict(model_treated, newdata = data.frame(index = cutoff)) -
  predict(model_control, newdata = data.frame(index = cutoff))
treatment_effect
```

```
1
-22.10483
```

```
## McCracy test
density_test <- rddensity(X = df$index, c = cutoff)
summary(density_test)
```

Manipulation testing using local polynomial density estimation.

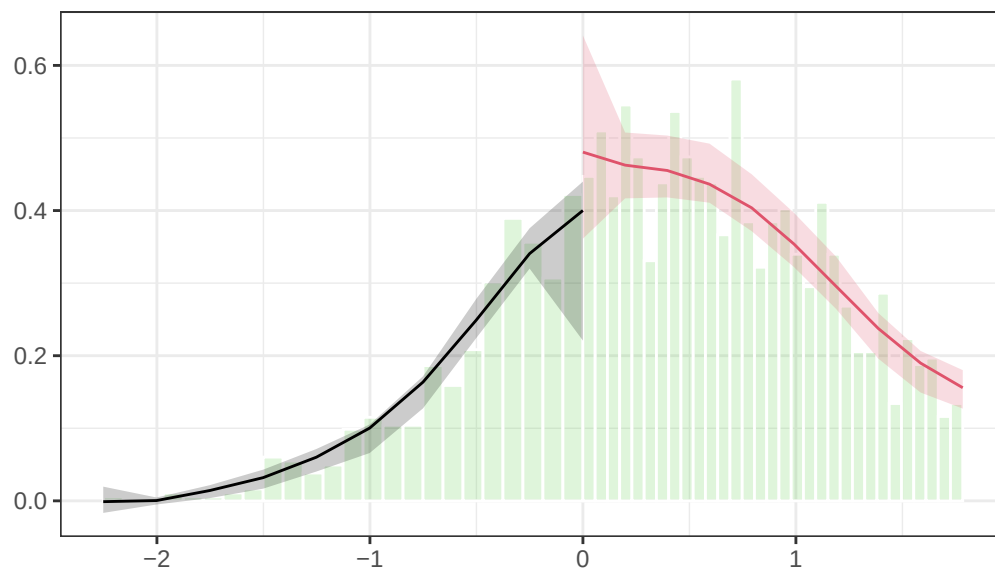
```
Number of obs =      1944
Model =           unrestricted
```

Kernel =	triangular	
BW method =	estimated	
VCE method =	jackknife	
c = 0	Left of c	Right of c
Number of obs	549	1395
Eff. Number of obs	425	536
Order est. (p)	2	2
Order bias (q)	3	3
BW est. (h)	0.751	0.594
Method	T	P > T
Robust	1.8424	0.0654

P-values of binomial tests ($H_0: p=0.5$).

Window Length / 2	<c	>=c	P> T
0.020	20	21	1.0000
0.039	35	31	0.7122
0.059	49	52	0.8424
0.078	64	72	0.5485
0.098	77	89	0.3933
0.117	90	107	0.2542
0.137	104	127	0.1476
0.156	113	143	0.0697
0.176	125	155	0.0829
0.195	143	181	0.0397

```
rdplotdensity(rdd = density_test, X = df$index)
```



\$Est1

Call: lpdensity

Sample size		549
Polynomial order for point estimation	(p=)	2
Order of derivative estimated	(v=)	1
Polynomial order for confidence interval	(q=)	3
Kernel function		triangular
Scaling factor		0.282552753474009
Bandwidth method		user provided

Use summary(...) to show estimates.

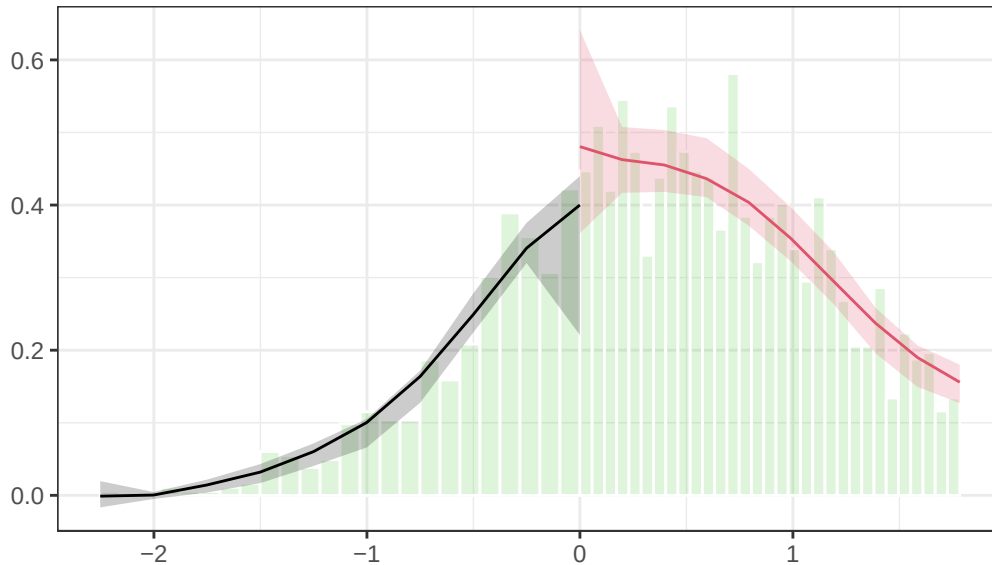
\$Estr

Call: lpdensity

Sample size		1395
Polynomial order for point estimation	(p=)	2
Order of derivative estimated	(v=)	1
Polynomial order for confidence interval	(q=)	3
Kernel function		triangular
Scaling factor		0.717961914565105
Bandwidth method		user provided

Use `summary(...)` to show estimates.

`$Estplot`



We estimate that the participation in the program reduced food consumption by 22.1 units in the first year after the intervention. We can repeat the estimation using the `rdd` package, which yields us an estimate as well as a confidence band calculated according to the formulas presented in the book. We look at all four targets.

```
result <- c()
for (outcome in c("conspcfood_t1", "conspcnnonfood_t1", "conspcfood_t2", "conspcnnonfood_t2")) {
  rdd_result <- rdrobust(df[[outcome]], df$index, c = cutoff, rho = 1,
                        weights = df$weights)
  result <- rbind(result, c(rdd_result$coef[1], rdd_result$se["Robust", ]))
}
resframe <- as.data.frame(result)
colnames(resframe) <- c("LATE", "s.e.")
rownames(resframe) <- c("Food T_1", "Non-Food T_1", "Food T_2", "Non-Food T_2")
print(resframe)
```

	LATE	s.e.
Food T_1	-22.170185	27.45233
Non-Food T_1	-9.126375	21.89831

Food T_2	54.886342	48.11563
Non-Food T_2	43.759831	32.34627

Estimation with Covariates

For identification and estimation of the average treatment effect at the cutoff value no covariate information is required except the running variable, but nevertheless in many applications additional covariates are collected which might be exploited for the analysis for efficiency reasons. The standard approach is simply to take up the regressors in the weighted least squares regression.

```
## specify linear model
model_treated <- lm(conspcfood_t1 ~ index + hhowhouse + headage + heademp + headeduc,
                    data = df[df$index > 0, ], weights = weights[df$index > 0])
model_control <- lm(conspcfood_t1 ~ index + hhowhouse + headage + heademp + headeduc,
                    data = df[df$index < 0, ], weights = weights[df$index < 0])
prediction_treated <- predict(model_treated, newdata = data.frame(
  index = cutoff,
  hhowhouse = weighted.mean(df[df$index > 0, ]$hhowhouse, w = weights[df$index > 0]),
  headage = weighted.mean(df[df$index > 0, ]$headage, w = weights[df$index > 0]),
  heademp = weighted.mean(df[df$index > 0, ]$heademp, w = weights[df$index > 0]),
  headeduc = weighted.mean(df[df$index > 0, ]$headeduc, w = weights[df$index > 0])
))
prediction_control <- predict(model_control, newdata = data.frame(
  index = cutoff,
  hhowhouse = weighted.mean(df[df$index < 0, ]$hhowhouse, w = weights[df$index < 0]),
  headage = weighted.mean(df[df$index < 0, ]$headage, w = weights[df$index < 0]),
  heademp = weighted.mean(df[df$index < 0, ]$heademp, w = weights[df$index < 0]),
  headeduc = weighted.mean(df[df$index < 0, ]$headeduc, w = weights[df$index < 0])
))
treatment_effect <- prediction_treated - prediction_control
treatment_effect
```

1
-23.78052

```

## rdrobust can also include covariates
result <- c()
for (outcome in c("conspcfood_t1", "conspcnonfood_t1", "conspcfood_t2", "conspcnonfood_t2")) {
  rdd_result <- rdrobust(df[[outcome]], df$index, c = cutoff, rho = 1, covs = df[, c(1:22)])
  result <- rbind(result, c(rdd_result$coef[1], rdd_result$se["Robust", ]))
}
resframe_adj <- as.data.frame(result)
colnames(resframe_adj) <- c("LATE", "s.e.")
rownames(resframe_adj) <- c("Food T_1", "Non-Food T_1", "Food T_2", "Non-Food T_2")
resframe_adj["% reduction"] <- (resframe_adj["s.e."] - resframe[, 2]) * 100 / resframe[, 2]
print(resframe_adj)

```

	LATE	s.e.	% reduction
Food T_1	-29.359238	21.86341	-20.358650
Non-Food T_1	-6.494893	20.61773	-5.847859
Food T_2	55.585665	44.35862	-7.808290
Non-Food T_2	45.439977	29.75665	-8.005925

Estimation of the Nuisance Function by Machine Learning

```

## running Variable and Outcome
df_ml <- df
investigated_outcome <- "conspcfood_t2"
names(df_ml)[names(df_ml) == "index"] <- "X"
names(df_ml)[names(df_ml) == investigated_outcome] <- "Y"

## baseline covariates including consumption
b_covs <- names(df_ml[, c(1:8, 10:17, 19, 22)])

## fixed effects for localities
i_fe <- as.matrix(dummy_cols(df_ml$clus, remove_first_dummy = TRUE))

## flexible covariates including localities indicators
f_covs <- as.matrix(model.matrix(~ .^2, data = df_ml[b_covs]))[, -1]

```



```
## we also define a bandwidth using IK
```

```
h_fs <- 2 * h
```

```
first_stage <- function() {
```

```
  ## set up the cross-fitting
```

```
  n <- nrow(df_ml)
```

```
  Kf <- 5 # number of folds
```

```
  sampleframe <- rep(1:Kf, ceiling(n / Kf))
```

```
  cfgroup <- sample(sampleframe, size = n, replace = FALSE)
```

```
  ## matrix to store eta predictions
```

```
  eta_fit <- matrix(NA, n, 3)
```

```
  ## create vector of observations to be considered in the first stage model
```

```
  weights <- (abs(df_ml$X) < h_fs)
```

```
  for (k in 1:Kf) {
```

```
    ## the fold to be predicted
```

```
    fold <- (cfgroup == k)
```

```
    data_fold <- df_ml[fold, ]
```

```
    ## the remaining folds used in training
```

```
    data_treated <- df_ml[which(df_ml$X > 0 & !fold & weights > 0), ]
```

```
    data_control <- df_ml[which(df_ml$X < 0 & !fold & weights > 0), ]
```

```
    ## specify the model
```

```
    model <- as.formula(paste("Y~", paste(b_covs, collapse = "+")))
```

```
    ## random forest
```

```
    rf1 <- randomForest(model, data = data_treated, mtry = 4, ntree = 1000)
```

```
    rf0 <- randomForest(model, data = data_control, mtry = 4, ntree = 1000)
```

```
    eta_fit[fold, 1] <- (predict(rf1, data_fold) + predict(rf0, data_fold)) / 2
```

```
  ## gradient bootsting
```

```

gbm1 <- gbm(model,
  data = data_treated, n.trees = 100, interaction.depth = 1,
  shrinkage = .1, distribution = "gaussian"
)
gbm0 <- gbm(model,
  data = data_control, n.trees = 100, interaction.depth = 1,
  shrinkage = .1, distribution = "gaussian"
)
eta_fit[fold, 2] <- (predict(gbm1, data_fold, n.trees = 100) + predict(gbm0, data_f

## lasso
las_base1 <- rlasso(model, data = data_treated)
las_base0 <- rlasso(model, data = data_control)
eta_fit[fold, 3] <- (predict(las_base1, data_fold) + predict(las_base0, data_fold))
}
return(eta_fit)
}

eta_fit <- first_stage()

```

```

methods <- c("Random Forest", "Gradient Boosting", "Lasso")

## create the residualized Y
second_stage <- function(eta_fit) {
  adj_results <- NULL

  for (i in seq_along(methods)) {
    m_y <- df_ml$Y - eta_fit[, i]
    rdd_result <- rdrobust(m_y, df$index, c = cutoff, rho = 1)
    adj_results <- rbind(adj_results, c(rdd_result$coef[1], rdd_result$se["Robust", ]))
  }
  return(adj_results)
}

## print results
adj_frame <- as.data.frame(second_stage(eta_fit))

```

```
rownames(adj_frame) <- methods  
colnames(adj_frame) <- c("LATE", "s.e.")  
print(adj_frame)
```

	LATE	s.e.
Random Forest	53.52256	46.13622
Gradient Boosting	53.79650	46.36468
Lasso	53.10464	46.75527